

Name and Surname: **GUGULETHU DLADLA**Student ITS No: **402413242**Qualification: **B S c I T** Year of Study: **3RD YEAR** Semester: **3RD SEMESTER**Assignment due date: **30 September**

Date submitted:

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.

Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____

Date: _____

Table of Contents

Problem Statement	3
Solution Overview & Target Audience	3
System Design	4
Screenshots of Outputs	6
Key Code Snippet.....	8
Challenges and Solution	10
References.....	11

Problem Statement

Household food waste is a widespread issue that may be caused by the lack of tracking of the existing grocery items, which leads to forgotten ingredients that may be expiring, redundant grocery purchases. This has users often struggle to decide what meals they can prepare using the only ingredient currently available in their kitchen through tracking system.

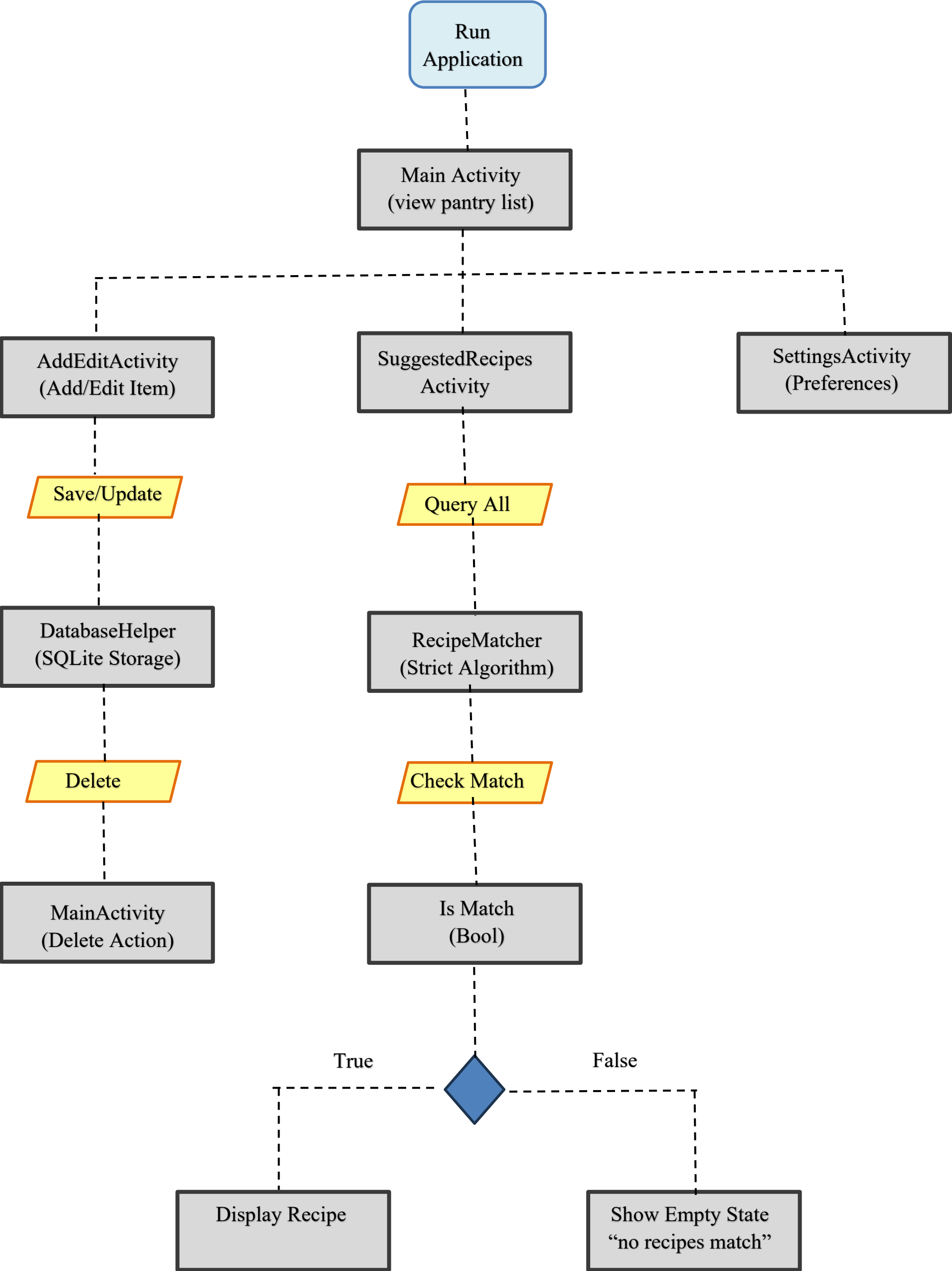
Solution Overview & Target Audience

The smart Pantry manager is an offline first android application that was emulated using 'small device' from device management. This application is catering to the families and individuals that track their local pantry inventory in real time. By storing ingredients quantities and expiration dates and applying a strict ingredient algorithm program. The app removes mealtime guesswork and reduces food waste and shows users what exactly they can prepare with what they have.

Core functionality

- Local Persistence Engine: Using android native frameworks such as SQLiteOpenHelper, the application creates a persistent local database "SmartPantry.db"
- Full CRUD operations: Users will be able to complete operations such as Create, Read, Update, and Delete items within the system as the system can perform those tasks.
- Pre-seeded Recipe Repository: The application will have a dedicated recipe list of 15 pre-loaded recipes.

System Design



Data Model ER Diagram

Table: Pantry			
Field Name	Data Types	Key type	Constraints
id	integer	PK	AutoIncrement
name	text		NOT NULL
quantity	real		NOT NULL
unit	text		NOT NULL
expiry_date	text		NOT NULL

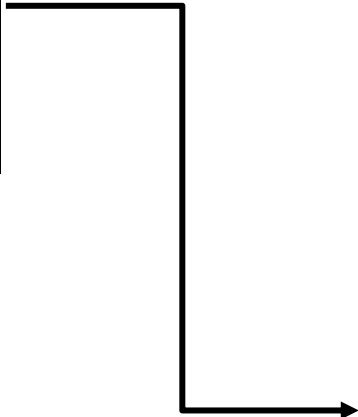
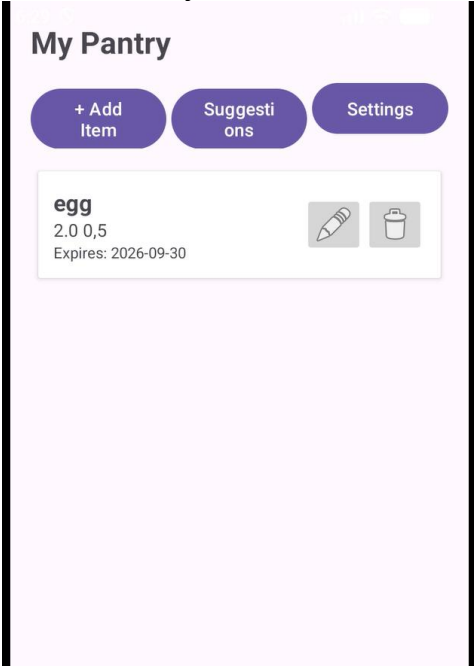


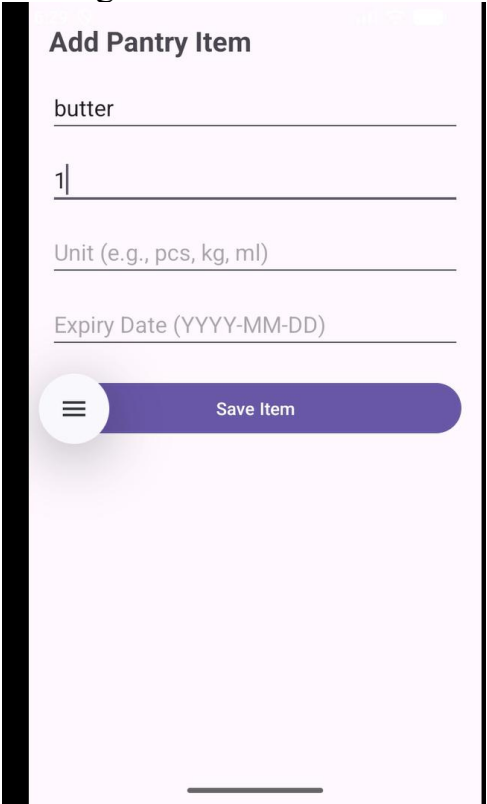
Table: Recipe			
Field Name	Data Types	Key type	Constraints
id	integer	PK	AutoIncrement
name	text		NOT NULL
ingredients	real		NOT NULL
steps	text		NOT NULL

Screenshots of Outputs

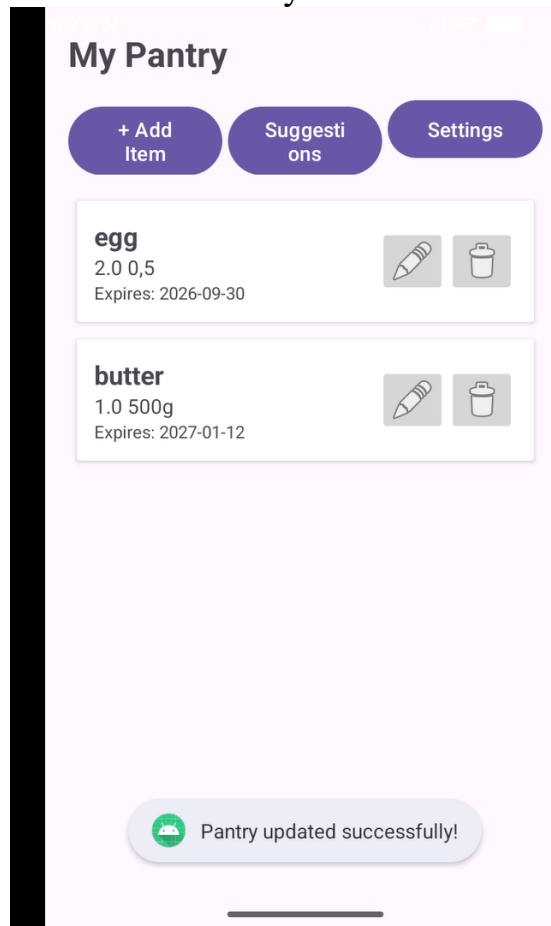
Main activity



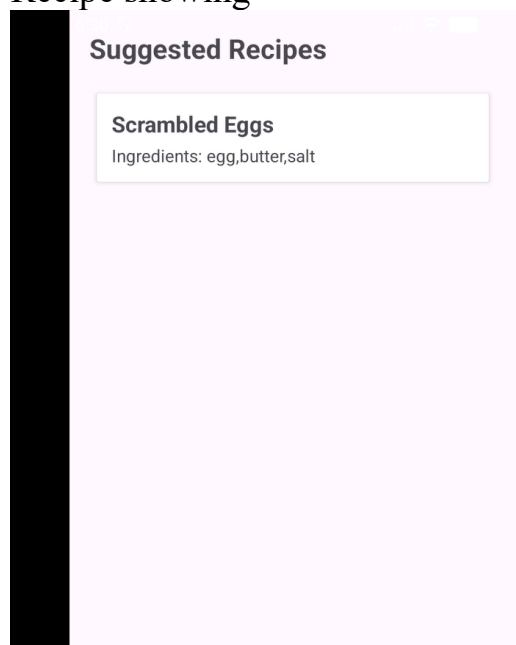
Adding item window



Added successfully



Recipe showing



Key Code Snippet

//this is taken from the DatabaseHelper.java

@Override

```
public void onCreate(SQLiteDatabase db) {  
    String createPantry = "CREATE TABLE " + TABLE_PANTRY + " (" +  
        COL_PANTRY_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +  
        COL_PANTRY_NAME + " TEXT, " +  
        COL_PANTRY_QTY + " REAL, " +  
        COL_PANTRY_UNIT + " TEXT, " +  
        COL_PANTRY_EXPIRY + " TEXT)";  
  
    String createRecipes = "CREATE TABLE " + TABLE_RECIPES + " (" +  
        COL_RECIPE_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +  
        COL_RECIPE_NAME + " TEXT, " +  
        COL_RECIPE_INGREDIENTS + " TEXT, " +  
        COL_RECIPE_STEPS + " TEXT)";  
  
    db.execSQL(createPantry);  
    db.execSQL(createRecipes);  
    seedInitialRecipes(db);  
}
```

This block code creates a local SQLITE table that on the app setup and pre-populates the database within the 15 initial recipe seeds needed on the first run.

//snippet two

// this block of code is from RecipeMatcher.java

```
private static boolean hasAllIngredients(List<PantryItem> pantryItems, Recipe
recipe) {
    List<String> requiredIngredients = recipe.getRequiredIngredientsList();
    if (requiredIngredients == null || requiredIngredients.isEmpty()) return false;

    for (String required : requiredIngredients) {
        boolean found = false;
        for (PantryItem pantryItem : pantryItems) {
            if (pantryItem.getName().trim().equalsIgnoreCase(required.trim())) {
                found = true;
                break;
            }
        }
        if (!found) return false; // Exclude recipe if even one ingredient is missing
    }
    return true;
}
```

This evaluate every required ingredient in a recipe against the users active pantry. Although if any required item is missing the recipe is then rejected.

//block code is from MainActivity.java

@Override

```
protected void onResume() {
    super.onResume();
    loadPantryData();
}
```

This code overrides the onResume() to ensure that the recyclerView is reflecting changes made in AddEditActivity.

Challenges and Solution

Delimiter mismatch in recipe ingredients:

Problem; Seeded recipe strings initially used to pipe (|) character delimiters, while recipe passing functions splits strings using commas. This caused the mismatch to evaluate ingredients as a single string resulting to 0 suggested recipe matches.

The solution was to update the DatabaseHelper.java Failed to use, separated ingredients list, bumped Database_Version to two to trigger onUpgrade() and verified getRequiredIngredientsList() to split strings elements accurately

Column lookup mismatch in recipe

Problem; RecipeMatcher.java referenced hardcoded column names ("title", "instructions") that was different from DatabaseHelper table constants ("name", "steps"), which then resulted in cursor indexing errors returning -1.

The solution was to refactor cursor queries in RecipeMatcher.java to directly reference DatabaseHelper public constants (DatabaseHelper.COL_RECIPE_NAME, DatabaseHelper.COL_RECIPE_STEPS).

Emulator System Timeout on Low-Resource Hardware:

Problem; Running a high-density x86 Pixel 7 emulator resulted to a response of ("Process system isn't responding").

Solution: Switched hardware profile in Device Manager to a low-density Small Phone profile running API 30 with RAM limited to 1024 MB.

Conclusion and reflection

Building the Smart Pantry Manager provided valuable hands-on experience with local SQLite persistence (SQLiteOpenHelper), dynamic Java adapters, activity lifecycle management, and strict relational logic.

References

- Andriod developers. *Create dynamic lists with RecyclerView* . n.d.
<https://developer.android.com/develop/ui/views/layout/recyclerview>. 22 september 2026.
- Developers. *Save data using SQLite*. n.d. https://developer.android.com/training/data-storage/sqlite?utm_source=gemini. 19 Septer 2026.
- oracle. *Class String*. n.d.
https://docs.oracle.com/javase/8/docs/api/java/lang/String.html?utm_source=gemini. 22 September 2026.